

SkyJEPA architecture and implementation notes

Arsh Noman

October 2026

1 Core Idea of SkyJEPA

SkyJEPA is a learned world model designed for long-horizon quadrotor prediction and model-based control. Its main architectural idea is to avoid directly predicting future physical states autoregressively. Instead, it learns a latent representation of the system dynamics and predicts how that latent representation evolves under future control actions.

The overall pipeline can be summarized as:

Trajectory Data $\rightarrow$ Latent Dynamics Model
 $\rightarrow$ Physics-Inspired Prober $\rightarrow$ Physical State Rollout
 $\rightarrow$ Sampling-Based Controller

For our toy implementation, the first objective should be to reproduce the **latent dynamics model**. The physics-inspired prober and controller can be added later.

2 Why SkyJEPA Uses Latent Prediction

The paper states:

“Rather than reconstructing future states or observations autoregressively, our model learns to predict future representations in a structured latent space.”

Earlier, the authors explain the problem with conventional *autoregressive world models*:

“Since the model is repeatedly conditioned on its own imperfect outputs, small one-step errors arising from uncertainty, approximation, or inaccuracies can accumulate over time, leading to drift, instability, and physically implausible trajectories.”

A conventional learned dynamics model might learn:

$$x_{t+1} = f(x_t, a_t)$$

and then recursively predict:

$$\hat{x}_{t+2} = f(\hat{x}_{t+1}, a_{t+1}), \quad \& \quad \hat{x}_{t+3} = f(\hat{x}_{t+2}, a_{t+2}), \quad \text{and so on.}$$

The problem is that every prediction contains some error. Because the next prediction depends on the previous predicted state, these errors can accumulate over long horizons. SkyJEPA instead learns:

$$s_t = \text{Enc}(X_t)$$

where s_t is a learned latent representation of the system, and predicts:

$$\tilde{s}_{t+1}$$

rather than directly predicting the entire physical state x_{t+1} . The central idea is therefore that a JEPA-based system must *predict future representations* rather than *reconstruct future observations*.

For our toy implementation, this distinction is fundamental. We are primarily building a **latent dynamics predictor**.

3 Overall SkyJEPA Architecture

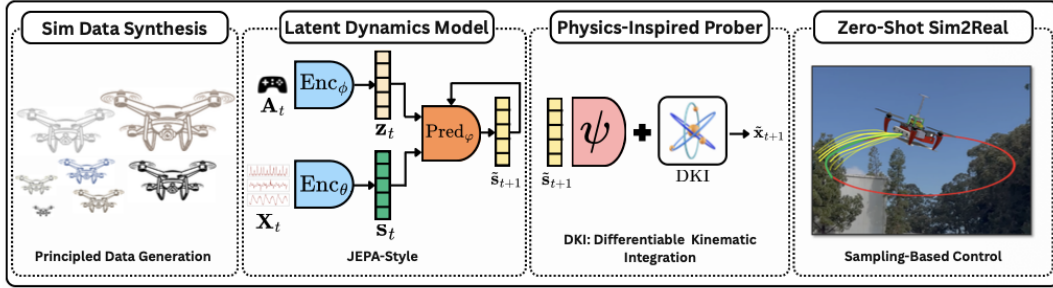


Figure 2: Overview of the proposed framework. We learn a JEPA-style latent dynamics model with a physics-inspired prober that maps abstract embeddings to physically meaningful states, enabling stable long-horizon quadrotor prediction. Trained entirely on domain-randomized simulation data and deployed within a sampling-based controller, the model achieves real-time execution and robust zero-shot sim-to-real transfer validated in outdoor flight experiments.

Figure 1: The above figure can be found on page 4 of the paper.

“We learn a JEPA-style latent dynamics model with a physics-inspired prober that maps abstract embeddings to physically meaningful states, enabling stable long-horizon quadrotor prediction.”

The architecture can be divided into four major components:

Data → JEPA Latent Dynamics → Physics Prober → Controller

More specifically,

1. State and action histories are encoded into latent representations.
2. A recurrent predictor evolves the latent state forward under future actions.

3. A physics-inspired prober converts latent predictions into interpretable physical trajectories.
4. MPPI (Model Predictive Path Integral) uses those predicted trajectories to choose control actions.

The distinctive part of SkyJEPA is primarily the first two components. MPPI is a downstream controller that uses the learned world model.

4 Input to the World Model: State and Action Histories

The paper defines:

$$X_t = [x_{t-H}^\top \quad \cdots \quad x_t^\top]^\top, \quad A_t = [a_{t-H}^\top \quad \cdots \quad a_t^\top]^\top$$

The authors explain:

“This history-based formulation gives the model access to recent motion and actuation trends.”

They further state that this helps account for

“actuator delay, drag, sensor noise, and platform-dependent dynamics.”

SkyJEPA does not only observe the current state x_t . Instead, it receives a window containing the recent states $X_t = [x_{t-H}, \dots, x_t]$ and recent actions $A_t = [a_{t-H}, \dots, a_t]$. This gives the network information about how the system has recently been moving and how it has recently been controlled. A single instantaneous observation may not reveal effects such as the ones stated above by the authors. For our toy model, the dataset should therefore eventually provide:

$$(X_t, A_t, \text{future states/actions})$$

rather than only independent transitions of the form:

$$(x_t, a_t, x_{t+1})$$

5 State Encoder and Action Encoder

The paper states:

“The state and action histories are encoded into latent representations $s_t = \text{Enc}_\theta(X_t)$ and $z_t = \text{Enc}_\phi(A_t)$.”

There are two separate encoders. The state-history encoder computes:

$$s_t = \text{Enc}_\theta(X_t)$$

where s_t is a compact representation of the recent state trajectory. The action-history encoder computes:

$$z_t = \text{Enc}_\phi(A_t),$$

where z_t represents recent control inputs. Conceptually,

$$\boxed{X_t \xrightarrow{\text{State Encoder}} s_t} \quad \text{and} \quad \boxed{A_t \xrightarrow{\text{Action Encoder}} z_t}$$

The latent vector s_t should contain the information necessary for predicting how the system will evolve. The latent vector z_t tells the predictor how the control inputs influence that evolution.

6 The Latent Dynamics Predictor

The paper states:

“The latent dynamics predictor then estimates the next latent state as $\tilde{s}_{t+1} = \text{Pred}_\varphi(s_t, z_t)$.”

For a prediction horizon T , the predictor is recursively unrolled:

$$\tilde{s}_{t+T} = \text{Pred}_\varphi(\cdots \text{Pred}_\varphi(\text{Pred}_\varphi(s_t, z_t), z_{t+1}), \dots, z_{t+T-1})$$

The predictor learns the transition dynamics in latent space:

$$(s_t, z_t) \rightarrow \tilde{s}_{t+1}$$

It is then recursively applied:

$$s_t \rightarrow \tilde{s}_{t+1} \rightarrow \tilde{s}_{t+2} \rightarrow \cdots \rightarrow \tilde{s}_{t+T}$$

Each transition is conditioned on the corresponding future action representation. The architecture can therefore be summarized as:

$$X_t \xrightarrow{\text{Enc}_\theta} s_t$$

$$A_t \xrightarrow{\text{Enc}_\phi} z_t$$

$$(s_t, z_t) \xrightarrow{\text{Pred}_\varphi} \tilde{s}_{t+1}$$

The predictor is essentially the learned dynamics model. In the actual SkyJEPA implementation, this predictor is a GRU.

7 What Makes the Training JEPA-Style?

The authors describe the objective as enforcing

“predictive consistency between rolled-out latent predictions and encoded future states.”

The prediction objective is

$$\mathcal{L}_{\text{pred}} = \frac{1}{T} \sum_{k=1}^T \|\tilde{s}_{t+k} - s_{t+k}\|_2^2.$$

During training, there are two representations of the future. First, the predictor produces

$$\boxed{\tilde{s}_{t+k}}$$

by recursively imagining the future in latent space. Second, the real future states from the dataset are passed through the state encoder:

$$\boxed{s_{t+k} = \text{Enc}_{\theta}(X_{t+k})}$$

These encoded future states become the targets. Training then makes

$$\tilde{s}_{t+k} \approx s_{t+k}$$

This is the central JEPA concept:

$$\boxed{\text{predict the representation of the future, not the future observation itself}}$$

There is no requirement that the latent representation reconstruct the original observation. For our toy implementation, this should be the first major training objective we reproduce.

8 Preventing Representation Collapse

The authors warn:

“The encoder could map all inputs to nearly constant embeddings, yielding low prediction error without preserving meaningful system dynamics.”

To prevent this, SkyJEPA uses Sketched Isotropic Gaussian Regularization (SIGReg). The final objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \mathcal{L}_{\text{SIGReg}}$$

A trivial solution to the JEPA objective would be

$$\text{Enc}(X) = c$$

for every possible input X , where c is the same constant vector. Then:

$$s_t = s_{t+1} = s_{t+2} = \dots = c$$

and the predictor could simply output c every time. This would produce nearly zero prediction error while learning nothing useful. SIGReg prevents this collapse by encouraging the distribution of latent embeddings to resemble an isotropic Gaussian. Conceptually,

$$\mathcal{L}_{\text{pred}} = \text{make future representations predictable}$$

whereas SkyJEPA’s solution is conceptually as follows:

$$\mathcal{L}_{\text{SIGReg}} = \text{prevent those representations from becoming trivial}$$

For our first toy implementation, it may be useful to initially verify that the latent prediction pipeline works before reproducing the complete SIGReg implementation.

9 The Two-Stage Training Pipeline

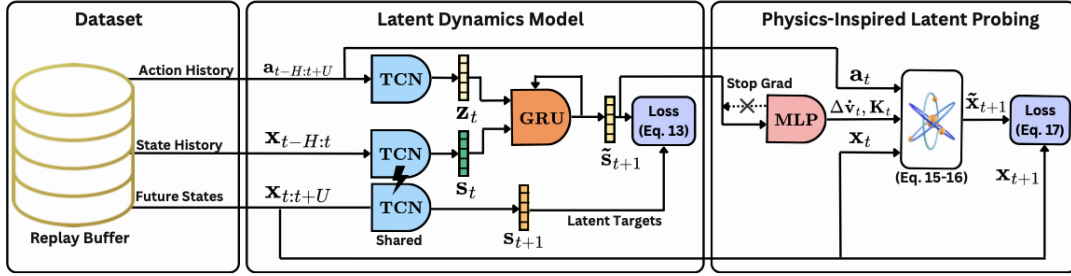


Figure 3: Two-stage training pipeline of our proposed approach. In the first stage, an encoder maps a history of past states into a latent representation, and a predictor propagates this latent forward in time conditioned on a sequence of control actions; in the second stage, a physics-inspired prober is trained on frozen latent embeddings to map them to physically grounded states.

Figure 2: The above figure can be found on page 5 of the paper.

The caption of the above figure states:

“In the first stage, an encoder maps a history of past states into a latent representation, and a predictor propagates this latent forward in time conditioned on a sequence of control actions; in the second stage, a physics-inspired prober is trained on frozen latent embeddings to map them to physically grounded states.”

Therefore, this provides us with the understanding that SkyJEPA is trained in two distinct stages. These stages are described as follows:

Stage 1: Learn the Latent Dynamics

$$X_t \xrightarrow{\text{State Encoder}} s_t$$

$$A_t \xrightarrow{\text{Action Encoder}} z_t$$

$$(s_t, z_t) \xrightarrow{\text{Predictor}} \tilde{s}_{t+1:t+T}$$

Future real states are separately encoded:

$$X_{t+k} \xrightarrow{\text{shared State Encoder}} s_{t+k}$$

The latent rollout is trained to match those future latent targets.

Stage 2: Learn Physical State Recovery

After Stage 1, the JEPA model is frozen:

$$\boxed{\text{Enc}_\theta, \text{Enc}_\phi, \text{Pred}_\varphi \text{ are frozen}}$$

The predicted latent states are then passed into the physics-inspired prober. Thus, the prober does not teach the JEPA representation. The latent dynamics are learned first.

10 Why Does SkyJEPA Need a Prober?

The paper states:

“For model-based control, latent predictions must be converted into physically meaningful quantities.”

The authors explain that this is necessary for

“evaluating tracking costs, enforcing state constraints, and respecting actuator limits.”

The latent vector s_t is useful for representing dynamics, but its individual dimensions have no guaranteed physical interpretation. For example, MPPI ultimately needs quantities such as

$$p, \quad v, \quad R, \quad \omega$$

to determine whether a candidate trajectory is good. A 24-dimensional latent vector alone cannot directly tell the controller *how far the drone is from the target?* or *whether an orientation constraint is violated?* The prober therefore creates the bridge:

$$\boxed{\text{Abstract Latent Dynamics} \rightarrow \text{Physical State Dynamics}}$$

11 The Physics-Inspired Prober

The prober predicts residual correction terms:

$$\{\Delta\dot{v}_{t+k}, K_{t+k}\} = \psi(\tilde{s}_{t+k})$$

The translational acceleration is then:

$$\dot{v}_t = \frac{\sum_{i=0}^3 f_{i,t}}{m} R_t e_3 - g + \Delta\dot{v}_t.$$

The state is propagated using:

$$p_{t+1} = p_t + v_t \Delta t,$$

$$v_{t+1} = v_t + \dot{v}_t \Delta t,$$

$$R_{t+1} = R_t \exp([\omega_t]_{\times} \Delta t),$$

$$\omega_{t+1} = \omega_t + \Delta\tau_t \Delta t$$

Thus, the prober does **not** simply decode:

$$\tilde{s}_t \rightarrow [p, v, R, \omega]$$

Rather, the model combines known physics with learned corrections. This can be described more simply as:

Known Kinematics + Learned Residual Dynamics

The known physics handles the structure that we already understand. The neural network only has to predict corrections for effects that the simple model does not capture well. This makes the prediction problem substantially easier than learning the complete physical dynamics from scratch.

12 Training the Physics Prober

The paper states:

“During prober training, a stop-gradient operation is applied to the predicted latent embeddings before they are passed to ψ .”

The purpose, according to the authors, is to:

“prevent the supervised state-recovery loss from modifying the learned latent dynamics.”

It has already been established that the prober objective is:

$$\mathcal{L}_{\text{prober}} = \frac{1}{T} \sum_{k=1}^T \|\tilde{x}_{t+k} - x_{t+k}\|_2^2$$

Given that structure, once the JEPA model has been trained its parameters should remain unchanged. The training flow becomes:

$$\tilde{s}_{t+k} \xrightarrow{\text{stop gradient}} \psi \rightarrow \tilde{x}_{t+k}$$

The prober is trained using physical trajectory error:

$$\tilde{x}_{t+k} \approx x_{t+k}$$

For our implementation, it would therefore make sense to separate training into two stages:

1. train the JEPA latent model,
2. freeze it and train the prober.

13 How SkyJEPA Is Used for Control

The paper states:

“We integrate our learned dynamics model within a sampling-based optimization framework, MPPI, where future action sequences are optimized using Monte Carlo sampling.”

For every sampled action sequence, the model recursively rolls out future latent states and then maps them into physical states using the prober.

This part is conceptually similar to the Cross-Entropy Method. The controller repeatedly considers possible future action sequences:

$$a_{t:t+T}^{(1)}, a_{t:t+T}^{(2)}, \dots, a_{t:t+T}^{(S)}$$

Each candidate sequence is passed through SkyJEPA:

Candidate Actions $\rightarrow$ Latent Rollout $\rightarrow$ Physical Rollout.

The predicted trajectory is then evaluated using a cost function. Thus,

Sample Candidate Actions $\rightarrow$ Imagine Futures $\rightarrow$ Evaluate Futures $\rightarrow$ Improve Actions

SkyJEPA itself is therefore not the controller. It is the **predictive engine used by the controller**. It can be described as MPPI effectively asking SkyJEPA *what will happen if I execute this sequence of motor commands?* and then choosing actions based on the predicted result.

14 Actual Neural Network Architecture

The paper specifies:

“Both Enc_θ and Enc_ϕ are implemented as Temporal Convolutional Networks (TCNs).”

The state encoder uses channel sizes

$$[8, 8, 16],$$

while the action encoder uses

$$[4, 4, 8]$$

The authors then state:

“Latent dynamics are modeled using a single-layer GRU predictor with hidden dimension 24.”

The history length is

$$H = 10$$

corresponding to 0.5 seconds at 20 Hz. The training rollout horizon is

$$T = 20$$

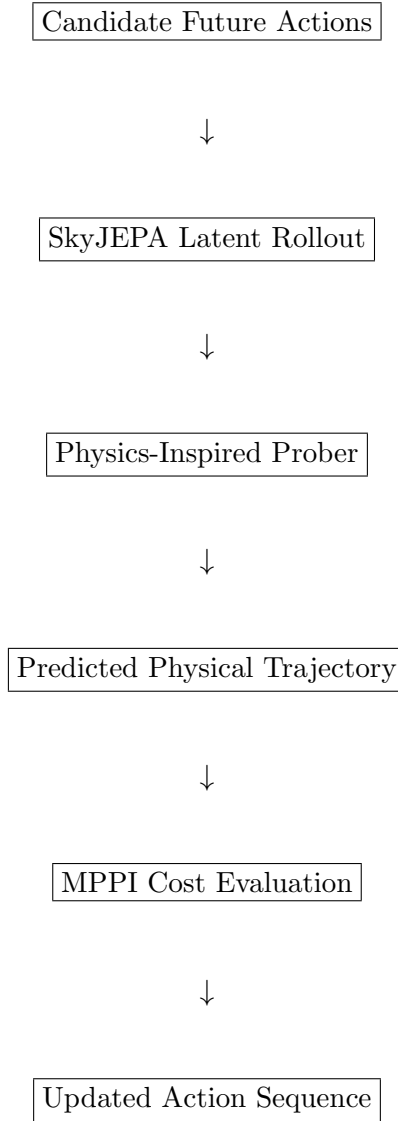
corresponding to 1.0 second. The actual neural architecture is surprisingly small. A simplified implementation looks like

```
StateEncoder = TCN
ActionEncoder = TCN
Predictor     = GRU
Prober       = MLP + Differentiable Physics
```

The paper reports that the latent dynamics model contains only approximately 9 000 parameters. The architectural novelty therefore does not come from using a very large Transformer or sophisticated neural backbone. The important innovation is what the model predicts and how the representation is trained which comes from the core of JEPA.

15 Runtime Control Pipeline

During deployment, the full system becomes



Only the first action is executed. The process is then repeated using the new observed state.

16 Our Initial Toy SkyJEPA Implementation

For our toy reimplementation, we should not attempt to reproduce the complete flight-control system immediately. A reasonable implementation sequence is:

Milestone 1: Dataset Windows

Generate trajectories containing $x_0, x_1, \dots, x_N$ and $a_0, a_1, \dots, a_N$. We must then construct samples containing

$$X_t, \quad A_t, \quad X_{t+1:t+T}, \quad A_{t+1:t+T}$$

Milestone 2: State Encoder

We then implement:

$$s_t = \text{Enc}_\theta(X_t)$$

A small TCN can approximately reproduce the paper architecture.

Milestone 3: Action Encoder

Implement:

$$z_t = \text{Enc}_\phi(A_t)$$

Milestone 4: Latent Predictor

Implement:

$$\tilde{s}_{t+1} = \text{Pred}_\varphi(s_t, z_t)$$

using a GRU. Then recursively produce:

$$\tilde{s}_{t+1}, \tilde{s}_{t+2}, \dots, \tilde{s}_{t+T}$$

Milestone 5: Future Latent Targets

Encode the real future state histories:

$$s_{t+k} = \text{Enc}_\theta(X_{t+k})$$

Then compute:

$$\mathcal{L}_{\text{pred}} = \frac{1}{T} \sum_{k=1}^T \|\tilde{s}_{t+k} - s_{t+k}\|_2^2$$

At this point, we have reproduced the central JEPA-style world-model idea.

Milestone 6: Anti-Collapse Regularization

Add SIGReg or an initially simplified anti-collapse mechanism such as constrastive learning. The objective becomes:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{regularization}}$$

Milestone 7: Simple Prober

Freeze

$$\text{Enc}_{\theta}, \quad \text{Enc}_{\phi}, \quad \text{Pred}_{\varphi}$$

Train a small network to recover interpretable physical quantities from the predicted latents. For the first toy implementation, this prober could initially be simpler than the full quadrotor physics-inspired mechanism.

Milestone 8: Physics-Inspired Prober

Replace the generic decoder with a structured model of the form:

$$\text{Known Dynamics} + \text{Learned Residual Corrections}$$

Milestone 9: Planning

Only after the world model is working should we integrate a planner such as MPPI or CEM.